Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Sistemas Operativos 1

Práctica No. 1 Fork(), Wait() y Exec()

Registro académico: 201831260
Nombre: Michael Kristopher Marín Reyes

Quetzaltenango, Febrero de 2026.

Ejercicio 1

```
--- EJERCICIO 1: CONVERSOR DE MONEDAS ---
Ingrese la cantidad en Quetzales (GTQ): 1000

Proceso Padre (PID: 56944) - Cantidad a convertir: 1000.00 GTQ
Creando procesos hijos...

=== HIJO 1 (PID: 56986, Padre: 56944) ===
=== HIJO 3 (PID: 56988, Padre: 56944) ===
Conversión a Euros (EUR):
1000.00 GTQ = 119.05 EUR (Tasa: 1 EUR = 8.40 GTQ)

Conversión a Dólares (USD):
=== HIJO 2 (PID: 56987, Padre: 56944) ===
Conversión a Pesos Mexicanos (MXN):
1000.00 GTQ = 2564.10 MXN (Tasa: 1 MXN = 0.39 GTQ)

1000.00 GTQ = 129.03 USD (Tasa: 1 USD = 7.75 GTQ)

Todos los procesos hijos han terminado.
```

```
--- EJERCICIO 1: CONVERSOR DE MONEDAS ---
Ingrese la cantidad en Quetzales (GTQ): 1000

Proceso Padre (PID: 56944) - Cantidad a convertir: 1000.00 GTQ
Creando procesos hijos...

=== HIJO 1 (PID: 57852, Padre: 56944) ===
Conversión a Dólares (USD):
1000.00 GTQ = 129.03 USD (Tasa: 1 USD = 7.75 GTQ)

=== HIJO 2 (PID: 57853, Padre: 56944) ===
Conversión a Pesos Mexicanos (MXN):
=== HIJO 3 (PID: 57854, Padre: 56944) ===
Conversión a Euros (EUR):
1000.00 GTQ = 119.05 EUR (Tasa: 1 EUR = 8.40 GTQ)

1000.00 GTQ = 2564.10 MXN (Tasa: 1 MXN = 0.39 GTQ)

Todos los procesos hijos han terminado.
Presione Enter para continuar...1
```

```
--- EJERCICIO 1: CONVERSOR DE MONEDAS ---
Ingrese la cantidad en Quetzales (GTQ): 1000

Proceso Padre (PID: 56944) - Cantidad a convertir: 1000.00 GTQ
Creando procesos hijos...

=== HIJO 1 (PID: 57883, Padre: 56944) ===
Conversión a Dólares (USD):
1000.00 GTQ = 129.03 USD (Tasa: 1 USD = 7.75 GTQ)

=== HIJO 2 (PID: 57884, Padre: 56944) ===
Conversión a Pesos Mexicanos (MXN):
1000.00 GTQ = 2564.10 MXN (Tasa: 1 MXN = 0.39 GTQ)

=== HIJO 3 (PID: 57885, Padre: 56944) ===
Conversión a Euros (EUR):
1000.00 GTQ = 119.05 EUR (Tasa: 1 EUR = 8.40 GTQ)

Todos los procesos hijos han terminado.
Presione Enter para continuar...
```

Preguntas

- **¿Está garantizado que los resultados siempre aparecerán en el orden Hijo 1, Hijo 2, Hijo 3 en la consola?**
  No está garantizado porque el scheduler del sistema operativo decide qué proceso se ejecuta primero y durante cuánto tiempo. Los procesos hijos compiten por el CPU y pueden ejecutarse en cualquier orden, incluso de manera intercalada. El orden de salida puede variar entre diferentes ejecuciones, como se puede observar en las imágenes.

- **¿Qué problema concreto ocurriría si esa validación se hiciera dentro del bloque del proceso hijo en lugar de hacerse en el padre?**
  Si la validación se hiciera en los hijos, ya se habrían creado procesos innecesariamente cuando el dato es inválido. Esto desperdicia recursos del sistema como memoria, tiempo de CPU, entradas en la tabla de procesos y complicaría la terminación del programa, ya que habría que asegurarse de que todos los hijos terminen correctamente.

- **¿Qué pasaría si uno de los procesos hijos modificará el valor de entrada que fue ingresado por el usuario? ¿Afectaría a los demás hijos?**
  No afectaría a los demás hijos debido al espacio de memoria independiente que tiene cada proceso. Cuando se crea un hijo con fork(), este obtiene una copia exacta del espacio de memoria del padre, pero cualquier modificación posterior en un hijo solo afecta a su propia copia. *Es lo que se conoce como "Copy-on-Write" (COW).*

Ejercicio 2

```
--- EJERCICIO 2: CIERRE DE CAJA ---
Ingrese el ingreso total del día (Q): 300

=== PROCESO PADRE (PID: 60404) ===
Ingreso total del día: Q300.00
Registro académico: 201831260

=== HIJO 1 (PID: 60415, Padre: 60404) - GASTOS OPERATIVOS ===
Desglose de gastos:
  Sueldo empleado: Q85.00 * 2 = Q170.00
  Electricidad: Q40.00 * 0 = Q0.00
  Agua estimada: Q15.00 * 1 = Q15.00
  Renta del local: Q480.00
  TOTAL GASTOS: Q665.00

=== HIJO 2 (PID: 60416, Padre: 60404) - GANANCIA NETA ===
Ingreso total: Q300.00
Gastos totales: Q245.00
Ganancia neta: Q55.00
RESULTADO: Con pérdida (ganancia < gastos)

=== PROCESO PADRE - RESUMEN DE CIERRE ===
Hijo 1 (Gastos) terminó con código: 0
Hijo 2 (Ganancia) terminó con código: 1
CIERRE DEL DÍA: Se registraron pérdidas
```

```
--- EJERCICIO 2: CIERRE DE CAJA ---
Ingrese el ingreso total del día (Q): 600

=== PROCESO PADRE (PID: 60404) ===
Ingreso total del día: Q600.00
Registro académico: 201831260

=== HIJO 2 (PID: 61222, Padre: 60404) - GANANCIA NETA ===
Ingreso total: Q600.00
Gastos totales: Q245.00
Ganancia neta: Q355.00
RESULTADO: Rentable (ganancia > gastos)
=== HIJO 1 (PID: 61221, Padre: 60404) - GASTOS OPERATIVOS ===
Desglose de gastos:
  Sueldo empleado: Q85.00 * 2 = Q170.00
  Electricidad: Q40.00 * 0 = Q0.00
  Agua estimada: Q15.00 * 1 = Q15.00
  Renta del local: Q480.00
  TOTAL GASTOS: Q665.00


=== PROCESO PADRE - RESUMEN DE CIERRE ===
Hijo 1 (Gastos) terminó con código: 0
Hijo 2 (Ganancia) terminó con código: 0
CIERRE DEL DÍA: Exitoso (hubo ganancia)
```

Preguntas

- **¿Por qué es importante declarar las variables en la clase padre y no en los procesos hijos?**
  Porque cuando se crea un hijo con fork(), este hereda una copia del espacio de memoria del padre en el momento de la creación. Si declaramos las variables después del fork() en cada hijo, cada hijo tendría su propia copia independiente y no podrían compartir información fácilmente. Además, declarar en el padre permite tener un punto central de control y facilita la comunicación entre procesos.

- **¿Cuál es la función de wait() y por qué es necesario usarla en el proceso padre dentro de un bucle?**
  wait() permite al proceso padre sincronizarse con la terminación de sus hijos, evitando que los hijos se conviertan en procesos zombies. Es necesario usarla en un bucle cuando hay múltiples hijos porque cada llamada a wait() solo espera a un hijo. El bucle asegura que el padre espere a TODOS sus hijos, capturando sus estados de salida y liberando los recursos asociados.

Ejercicio 3

```
=== PROCESO PADRE (PID: 60404) ===
Directorio a listar: /home/michael

=== HIJO (PID: 61704, Padre: 60404) - EJECUTANDO ls -l ===
total 19960
drwxr-xr-x  3 michael michael     4096 ago 21  2024  Android
drwxr-xr-x  2 michael michael     4096 jul 15  2025  AndroidStudioProjects
-rw-r--r--  1 michael michael    11909 ago 27 19:49  archivo-calificacion.csv
drwxr-xr-x  6 michael michael     4096 oct 22 16:26  Arduino
-rw-r--r--  1 michael michael  1244238 ene 30 11:35  backup_mysql.sql
-rw-r--r--  1 michael michael   231497 ene 29 15:07  backup_postgres.sql
-rwxr-xr-x  1 michael michael       47 sep 13  2024  Camara.sh
-rw-r--r--  1 michael michael     5533 feb 14  2024  consultas.txt
-rw-r--r--  1 michael michael 18256584 oct 22  2023  db-derby-10.17.1.0-bin.tar.gz
-rw-r--r--  1 michael michael     1150 feb 18 15:41  derby.log
drwxr-xr-x  2 michael michael    32768 feb 20 15:24  Descargas
drwxr-xr-x 31 michael michael     4096 feb 18 22:36  Documentos
drwxr-xr-x  5 michael michael     4096 nov 12  2023  env
drwxr-xr-x  3 michael michael     4096 ene 19 12:16  Escritorio
-rw-r--r--  1 michael michael   387526 nov 11 15:21  Escritorio.3mf
-rw-r--r--  1 michael michael    16226 oct 28 15:23  esquema.sql
drwxr-xr-x  5 michael michael     4096 feb 18 15:01  firstdb
drwxr-xr-x  9 michael michael    40960 dic 20 16:50  Imágenes
-rw-r--r--  1 michael michael      238 nov 12  2023  JuntarPdf.py
```

```
drwxr-xr-x  5 michael michael     4096 oct  9 22:06  simulador_venv
drwxr-xr-x  4 michael michael     4096 feb 18 15:41  sistemahuehue
drwxr-xr-x  4 michael michael     4096 feb 18 15:09  sistemahuhue
drwx------  5 michael michael     4096 mar  9  2025  snap
-rw-r--r--  1 michael michael      594 jun 14  2025  validaciones_aritmeticas.objc
drwxr-xr-x  6 michael michael     4096 feb 20 10:59  Videos
drwxr-xr-x  4 michael michael     4096 feb 19 22:40 'VirtualBox VMs'

=== PROCESO PADRE - RESULTADO ===
El proceso hijo se ejecutó correctamente
```

Preguntas

- **¿Por qué es necesario utilizar una función de la familia exec() en el proceso hijo para ejecutar el comando ls -l y qué sucede con el código del hijo después de llamar a exec() si se ejecuta correctamente?**
  Es necesario usar exec() porque queremos ejecutar un programa diferente (ls) desde nuestro proceso. exec() reemplaza completamente la imagen del proceso actual con la del nuevo programa. Si la llamada a exec() tiene éxito, no hay retorno - el código del proceso hijo es reemplazado por el código del comando ls, y nunca ejecuta las líneas posteriores a exec(). El proceso hijo se transforma en el proceso ls.

- ¿Cuál es la diferencia entre WIFEXITED() y WEXITSTATUS()?
  - WIFEXITED(status): Es un marco que evalúa si el proceso hijo terminó normalmente, es decir, llamando a exit(), retorna un valor distinto de cero si terminó normalmente.
  - WEXITSTATUS(status): Es una macro que solo si WIFEXITED es verdadero extrae el código de salida, el argumento de exit() o el valor retornado por main() del proceso hijo.

  En sí, estas macros trabajan en conjunto, donde primero verifica con WIFEXITED si el hijo terminó normalmente y luego WEXITSATATUS obtiene el código de salida.

Anexos

[Código utilizado]